Rastreo de objetos en Python

https://gemini.google.com/app/ba0513d85b987877

User prompt: Dame un programa en Python que a partir de una secuencias de imágenes jpg me de la posición de un objeto

Response: Aquí tienes una solución completa y modular en Python utilizando **OpenCV**. Este programa toma una secuencia de imágenes `.jpg` cargadas desde una carpeta y detecta la posición del centro de un objeto en cada fotograma.

Para lograr un rastreo robusto, el script utiliza **máscaras de color en espacio HSV**, que es el método más efectivo y computacionalmente liviano para segmentación basada en la vista.

Requisitos previos

Asegúrate de instalar OpenCV y NumPy en tu entorno:

```
pip install opencv-python numpy
```

Código en Python

```
import os
import glob
import cv2
import numpy as np

def rastrear_objeto_en_secuencia(
    carpeta_imagenes,
    color_hsv_bajo,
    color_hsv_alto,
    min_area=500,
    guardar_video=False,
    output_filename="resultado.mp4"
):
    """
    Lee una secuencia de imágenes JPG en una carpeta, detecta la posición de un
    objeto por su color y retorna una lista con las coordenadas (frame, x, y).
    """
    # 1. Obtener y ordenar la secuencia de imágenes
    patron = os.path.join(carpeta_imagenes, "*.jpg")
    rutas_imagenes = sorted(glob.glob(patron))

    if not rutas_imagenes:
        print(f"Error: No se encontraron archivos .jpg en '{carpeta_imagenes}'.")
        return []

    # Cargar primera imagen para obtener dimensiones
    primera_img = cv2.imread(rutas_imagenes[0])
    if primera_img is None:
        print("Error: No se pudo leer la primera imagen.")
        return []

    alto, ancho, _ = primera_img.shape
    posiciones = []

    # Configurar el guardado de video opcional
    writer = None
    if guardar_video:
        fourcc = cv2.VideoWriter_fourcc(*'mp4v')
        writer = cv2.VideoWriter(output_filename, fourcc, 30.0, (ancho, alto))

    print(f"Procesando {len(rutas_imagenes)} imágenes...")

    # 2. Procesar fotograma por fotograma
    for idx, ruta in enumerate(rutas_imagenes):
        frame = cv2.imread(ruta)
        if frame is None:
            continue

        # Convertir a espacio de color HSV
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # Crear máscara binaria para filtrar el color
```

```
            mask = cv2.inRange(hsv, color_hsv_bajo, color_hsv_alto)

            # Aplicar operaciones morfológicas para eliminar ruido (puntos aislados)
            kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
            mask = cv2.erode(mask, kernel, iterations=2)
            mask = cv2.dilate(mask, kernel, iterations=2)

            # Encontrar contornos en la máscara
            contornos, _ = cv2.findContours(mask, cv2.RETRATION_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

            centro_x, centro_y = None, None

            if contornos:
                # Seleccionar el contorno más grande (el objeto principal)
                c_max = max(contornos, key=cv2.contourArea)
                area = cv2.contourArea(c_max)

                # Filtrar si el área supera el umbral mínimo de detección
                if area > min_area:
                    # Calcular momentos de la figura para obtener el centroide (X, Y)
                    M = cv2.moments(c_max)
                    if M["m00"] != 0:
                        centro_x = int(M["m10"] / M["m00"])
                        centro_y = int(M["m01"] / M["m00"])

                        # Dibujar visualización sobre el fotograma
                        cv2.circle(frame, (centro_x, centro_y), 7, (0, 255, 0), -1)
                        cv2.putText(
                            frame,
                            f"Pos: ({centro_x}, {centro_y})",
                            (centro_x + 10, centro_y - 10),
                            cv2.FONT_HERSHEY_SIMPLEX,
                            0.5,
                            (0, 255, 0),
                            2
                        )

                        # Dibujar el contorno del objeto
                        cv2.drawContours(frame, [c_max], -1, (255, 0, 0), 2)

            posiciones.append({"frame": idx, "archivo": os.path.basename(ruta), "x": centro_x, "y": centro_y})

            # Escribir frame procesado al video de salida
            if writer:
                writer.write(frame)

        if writer:
            writer.release()
            print(f"Video resultante guardado como '{output_filename}'.")

        return posiciones

    # ========================================
    # EJEMPLO DE USO
    # ========================================
    if __name__ == "__main__":
        # 1. Ruta a la carpeta que contiene tus imágenes .jpg
        CARPETA = "./mi_secuencia_jpg"

        # 2. Definir el rango del color del objeto en HSV.
        # Ejemplo para un objeto ROJO:
        COLOR_BAJO = np.array([0, 120, 70])
        COLOR_ALTO = np.array([10, 255, 255])

        # Ejemplo para un objeto AZUL (descomentar si es azul):
        # COLOR_BAJO = np.array([100, 150, 0])
        # COLOR_ALTO = np.array([140, 255, 255])

        # 3. Ejecutar la función de detección
        resultados = rastrear_objeto_en_secuencia(
            carpeta_imagenes=CARPETA,
            color_hsv_bajo=COLOR_BAJO,
```

```
    color_hsv_alto=COLOR_ALTO,
    min_area=300,            # Área mínima en píxeles para considerar un objeto
    guardar_video=True       # Guarda un archivo mp4 visual
)

# 4. Imprimir los resultados de las primeras posiciones
print("\nCoordenadas encontradas (primeros 5 frames):")
for pos in resultados[:5]:
    print(f"Frame: {pos['frame']} | Archivo: {pos['archivo']} -> X: {pos['x']}, Y: {pos['y']}")
```

¿Cómo funciona el script?

1. **Carga secuencial:** Usa `glob.glob` combinado con `sorted()` para asegurar que las imágenes se lean en estricto orden alfabético/numérico (por ejemplo, `img_001.jpg`, `img_002.jpg`).
2. **Espacio HSV:** Convierte las imágenes de BGR a HSV (Hue, Saturation, Value), permitiendo aislar colores específicos de forma independiente a la variación de iluminación.
3. **Morfología de imágenes:** Aplica operaciones de *Erosión* y *Dilatación* para eliminar pequeños píxeles de ruido o falsos positivos.
4. **Cálculo del centroide:** A través de los **momentos de la imagen** (`cv2.moments`), obtiene el centro (x, y) de la forma geométrica delimitada.
